

Laboratorio Semana 8 – Cifrado AES-256, HMAC-SHA256, SSH y Git

Nombre: Mauricio David Lima Juárez

Objetivo Aplicar cifrado simétrico para proteger la confidencialidad de un archivo, utilizar HMAC-SHA256 como huella autenticada y trabajar con un repositorio compartido de GitLab mediante SSH.

Herramientas: PowerShell + Git + OpenSSL

Regla de seguridad: Nunca subas la contraseña o clave secreta al repositorio de GitLab.

1. Conexión al repositorio mediante SSH

Verifica Git: `git --version`

Prueba la conexión: `ssh -T git@github.com`

Si es la primera conexión, verifica el fingerprint del servidor antes de aceptarlo.

Clona el repositorio compartido: `git clone git@github.com:[tu_repo]_git`

Entra al repositorio y crea tu rama: `cd git checkout -b lab-[nombre_branch]`

2. Crear un archivo personalizado

Crea un archivo cuyo nombre incluya tu nombre y apellido. Cada estudiante debe utilizar contenido personalizado.

Ejemplo: `mensaje_LuisRojas.json`

Contenido:

```
{  
  "nombre": "TU NOMBRE",  
  "curso": "Manejo e Implementación de Archivos",  
  "carne": "[tu_carne]",  
  "archivo": "mensaje_[Nombre_apellido].txt"  
}
```

3. Cifrar el archivo con AES-256

Utiliza **OpenSSL** para cifrar tu archivo con **AES-256-CBC**. El archivo resultante tendrá extensión `.enc`

3.1 Verifica OpenSSL

Ejecuta el siguiente comando en **PowerShell**: `openssl version`

Si OpenSSL no está instalado, ejecuta: `winget install --id ShiningLight.OpenSSL.Light --exact`

Si instalaste OpenSSL, **reinicia VS Code** antes de continuar.

3.2 Comprueba instalación

`Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

3.3 Cifra tu archivo

Comprueba dónde quedó instalado: `Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

Agrega OpenSSL al PATH de tu usuario: `$opensslPath = "C:\Program Files\OpenSSL-Win64\bin"`

`[Environment]::SetEnvironmentVariable("Path", $env:Path + ";" + $opensslPath, "User")`

Actualiza el PATH de esta terminal inmediatamente: `$env:Path += ";C:\Program Files\OpenSSL-Win64\bin"`

3.4 Cifra tu archivo

Ejecuta: `openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.json -out .\mensaje_TuNombre.enc`

Cuando OpenSSL solicite la contraseña, introduce una contraseña que solo tú conozcas. Deberás introducirla dos veces. **NO escribas la contraseña en este laboratorio ni la subas a GitHub.**

3.5 Comprueba que se creó el archivo: Ejecuta: `Get-Childitem`

Deben aparecer, como mínimo: `mensaje_TuNombre.txt` `mensaje_TuNombre.enc`

3.6 Comprueba que el archivo cifrado no muestra directamente el mensaje:

`Get-Content .\mensaje_TuNombre.enc`

¿Qué observaste?

4. Descifrar y recuperar el archivo

Utiliza la misma contraseña para recuperar el contenido original.

`openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.enc -out .\mensaje_TuNombre_recuperado.json`

Comprueba el contenido recuperado: `Get-Content .\mensaje_TuNombre_recuperado.json`

5. Generar una huella autenticada con HMAC-SHA256

Ahora generarás una huella que depende del archivo y de una clave secreta. Esto permite comprobar integridad y autenticidad cuando quien verifica también conoce la clave.

archivo + clave secreta → HMAC-SHA256 → huella

corre: `openssl dgst -sha256 -hmac "TU_CLAVE_SECRETA" .\mensaje_LuisRojas.json`

¿Cuál es el HMAC-SHA256 obtenido?

`HMAC-SHA2-256(.\mensaje_MauricioLima.json)=`
`6c307dbcb3eb6917bc56692aac1c28c4f20a8eadbf00753b77e0e7b67071f51`

6. Hash vs. cifrado vs. HMAC

Mecanismo	Objetivo	Clave	Ejemplo
SHA-256	Integridad	No	Huella
AES-256 (encrypt)	Confidencialidad	Sí	Archivo .enc
HMAC-SHA256	Integridad + autenticidad	Sí	Huella autenticada
SSH	Comunicación/autenticación segura	Sí	GitLab

Pregunta 1: Si alguien obtiene tu archivo .enc pero no conoce la contraseña, ¿qué propiedad estás protegiendo?
Protege la confidencialidad de los datos.

Pregunta 2: ¿Por qué no debes subir la contraseña a GitLab? Porque GitLab es un control de versiones y cualquier contraseña subida al repositorio queda expuesta públicamente o accesible para cualquier colaborador con permisos.

Pregunta 3: ¿Qué diferencia principal existe entre SHA-256 y HMAC-SHA256? La diferencia principal es que **SHA-256** es una función hash criptográfica unidireccional que genera una huella digital única a partir de cualquier dato de entrada, mientras que **HMAC-SHA256** combina SHA-256 con una **clave secreta** compartida.

7. Entrega

Dentro del repositorio de GitHub, crea la carpeta semana-8/lab8/ y coloca en ella un PDF con la solución del laboratorio, incluyendo las respuestas, el HMAC-SHA256 obtenido y pantallazos como evidencia de los comandos y procedimientos realizados. También incluir el archivo .json creado y el archivo .enc generado durante el cifrado. Enviar en la plataforma el enlace de la carpeta lab8 como evidencia de la entrega.

```
PS C:\practica_URL> cd url-mia-2026
PS C:\practica_URL\url-mia-2026> cd semana_8
PS C:\practica_URL\url-mia-2026\semana_8> cd lab8
PS C:\practica_URL\url-mia-2026\semana_8\lab8> Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"
```

Directorio: C:\Program Files\OpenSSL-Win64\bin

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	25/08/2026 09:21	821760	openssl.exe

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> openssl enc -aes-256-cbc -salt -pbkd
```

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> $env:Path += ";C:\Program Files\OpenSSL-Win64\bin"
```

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> openssl version
OpenSSL 4.0.2 25 Aug 2026 (Library: OpenSSL 4.0.2 25 Aug 2026)
```

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_MauricioLima.json -out .\mensaje_MauricioLima.enc
enter AES-256-CBC encryption password:
```

Verifying - enter AES-256-CBC encryption password:

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> dir
```

Directorio: C:\practica_URL\url-mia-2026\semana_8\lab8

Mode	LastWriteTime	Length	Name
------	---------------	--------	------

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> dir
```

```
Directorio: C:\practica_URL\url-mia-2026\semana_8\lab8
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	28/08/2026 09:06	151	mensaje_MauricioLima.json

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_MauricioLima.json -out .\mensaje_MauricioLima.enc
enter AES-256-CBC encryption password:
```

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> Get-ChildItem
>>
```

```
Directorio: C:\practica_URL\url-mia-2026\semana_8\lab8
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	28/08/2026 09:17	176	mensaje_MauricioLima.enc
-a----	28/08/2026 09:06	151	mensaje_MauricioLima.json

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_MauricioLima.enc -out .\mensaje_Mauricio_recuperado.json
enter AES-256-CBC decryption password:
```

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> Get-Content .\mensaje_Mauricio_recuperado.json
{
  "nombre": "Mauricio Lima",
  "curso": "Manejo e Implementaci3n de Archivos",
  "carne": "[1080825]",
  "archivo": "mensaje_[Mauricio_Lima].txt"
}
```

```
PS C:\practica_URL\url-mia-2026\semana_8\lab8> openssl dgst -sha256 -hmac "TU_CLAVE_SECRETA" .\mensaje_MauricioLima.json
HMAC-SHA2-256(.\mensaje_MauricioLima.json)= 6c307dbcb3eb6917bc56692aaac1c28c4f20a8e adbf00753b77e0e7b67071f51
PS C:\practica_URL\url-mia-2026\semana_8\lab8>
```